Name :- Tailor Krima
Roll no :- IT26A50

Page No.
Date: / /

ASSIGNMENT

1) Introduction to Databases, Basic Definitions and Importance

=> A database is an organized collection of related data that is stored electronically and can be easily accessed, managed, updated and retrieved.

Eg :- A college database may store student information such as Student ID, Name, Course, Age, Email and Marks.

A Database Management System is software used to create, store, manage and retrieve data from a database.

Eg :- MySQL, Oracle

Importance of Databases :-

(i) Easy Data storage :- Large amounts of data can be stored systematically.

(ii) Quick Data Retrieval :- Required information can be searched and retrieved quickly.

(iii) Data Management :- Data can be easily added, modified or deleted.

(iv) Data Security :- Access to important data can be controlled using users and permissions.

(v) Data Accuracy :- Rules and Constraints help maintain correct and consistent data.

(vi) Data Sharing :- Multiple users or applications can access the same database.

(vii) Backup and Recovery :- Databases can be backed up and restored if data is lost or damaged.

(viii) Handles Large Amounts of Data :- Databases are designed to store and manage large amounts of information efficiently.

(ix) Supports Decision Making :- Databases can provide useful information and reports that help organizations make better decisions.

Eg :- Consider an online shopping applications.
A customer searches for a mobile phone.
Customer → shopping Application → DBMS → Database

(i) Customer searches for a product.

(ii) The application sends the request to the DBMS.

(iii) The DBMS searches the database.

(iv) Product information is retrieved.

(v) The information is displayed to the customer.

It may contain :-
Product ID → Product Name → Price → stock → Brand.
This process allows the customer to get information quickly.

2) An Example Database System ; Characteristics of DB Approach.

=) An Example of Database System :- College Database System.
Consider a college that needs to maintain information about students, teachers, courses, subjects, attendance and examination results.
Instead of maintaining separate files for every department, the college can use a centralized database.
Main tables :- Student, Course, Marks
How the system works :-
(i) The college stores student information in the database.
(ii) Teachers enter attendance and examination marks.
(iii) Students can view their marks and attendance.
(iv) Administrators can add, modify or delete student records.
(v) The DBMS manages all these operations.
(vi) Authorized users can access the required information.
Simple structure :-
Users → Applications → DBMS → Student
For example :-
student → College Portal → DBMS → Student Database

Characteristics of Database Approach :-

(1) Self-Describing Nature of Database System :-

- A database system contains not only the actual data but also information about the data.
  This additional information is called metadata.

(2) Program-Data Independence:-
- The structure of data is separated from the application programs that use it.
- This means changes to the database structure do not always require major changes to application programs.

(3) Data Abstraction.
- Database systems hide unnecessary implementation details from users.
- Users do not need to know:-
  - Where exactly the data is physically stored.
  - How it is stored on the disk.
  - How the database searches for the data.
- They simply interact with the database through an application or query.

(4) Support for Multiple Views of Data.
- Different users may need different information from the same database.
- A database can provide different views for different users.

(5) Data Sharing.
- A database allows multiple authorized users and applications to share the same data.

(6) Controlled Data Redundancy.
- It means unnecessary duplication of data.

(7) Data Consistency :-
- The database approach helps maintain consistent data.

(8) Data Security :-
- Database systems provide mechanisms to protect data from unauthorized access.
- Different users can be given different access permissions.

(9) Integrity Constraints :-
- A database can apply rules called integrity constraints to maintain the correctness of data.

(10) Backup and Recovery :-
- Database systems provide mechanisms for backup & recovery.

(11) Concurrency Control :-
- A database can be accessed by multiple users at the same time.
- It ensures that simultaneous operations do not create incorrect results.

3) Actors on the Scene, Workers behind the Scene.

=) In a database environment, many people are involved in designing, managing, developing and using the database system. They can broadly be divided into two groups :-

(i) Actors on the Scene :- People who directly interact with or manage the database system.

(ii) Workers Behind the Scene :- People who develop and maintain

the DBMS software and supporting tools.

1] Actors on the Scene :-

- Actors on the scene are the people who directly use, design, administer, or interact with the database system.

A] Database Administrators (DBAs) :-

↳ A DBA is responsible for all overall management and control of the database.

B] Database Designers :-

↳ They are responsible for designing the structure of the database.

C] End Users :-

↳ End users are people who use the database to perform their daily activities.

↳ They can be divided into different types :-

(i) Casual Users
(ii) Naive / Parametric Users
(iii) Sophisticated Users.
(iv) Stand-alone Users.

D] System Analysts :-

↳ A system Analysts studies the organizations requirements and determines what the database application should do.

E] Application Programmers :-

↳ They develop programs that allow users to interact with the database.

2] Workers Behind the scene.

- Workers Behind the scene are people who develop the DBMS and supporting software, rather than normally using the database for day-to-day work.

A] DBMS System Designers and Implementers:-

↳ These people design and develop the actual DBMS software.

B] Tool Developers:-

↳ Tool developers create software tools that make database development and management easier.

C] Operators and Maintenance Personnel:-

↳ These people are responsible for maintaining the hardware and software environment in which the database operates.

Online banking database system.

Eg:- Actors on the Scene:-

- DBA :- Maintains security and backup.
- Database Designers:- Designs customer, account & transaction tables.
- Bank Employee :- Enters & retrieves customer information.
- Customer:- checks balance and transactions.
- Application Programmer :- Develops the banking applications.
- System Analyst :- Collects banking requirements.

Workers Behind the Scene:-

- DBMS Developers :- Develop the database managment software.
- Tool Developers :- Develop database monitoring and administration tools.
- System Operators :- Maintain servers and supporting infrastructure.

4] Advantages of Using the DBMS Approach, History of DB Applications.

=) A DBMS is software that allows users to create, store, organize, retrieve, update and manage data in a database.

↳ Advantages of Using the DBMS Approach:-

1] Reduced Data Redundancy:-

↳ Data redundancy means unnecessary duplication of the same data.

2] Improved Data Consistency:-

↳ When duplicate data exists, one copy may be updated while another remains unchanged.

↳ A DBMS helps maintain Consistent and accurate data by reducing duplication and controlling updates

3] Data sharing:-

↳ A DBMS allows multiple authorized users and applications to access the same database.

4] Improved Data Security.

↳ A DBMS provides security mechanisms to prevent unauthorized access.

5] Data Integrity:-

↳ A DBMS allows integrity constraints to be defind to ensure

6] Data Independence:-

↳ Data Independence means that changes in the structure

or storage of data can be made with minimal changes to application programs.

5] Backup and Recovery :-
↳ A DBMS provides facilities for backup and recovery.
↳ If data is lost because of system failure, Hardware failure, Accidental deletion and software problems, the database can be restored from backups.

6] Concurrent Access :-
↳ Multiple users can access the database at the same time. The DBMS provides concurrency control to ensure that simultaneous operations do not cause incorrect results.

↳ History of Database Applications.
The development of database applications can be understood through different stages :-
1] File Processing Systems - 1960s
• In the early days, organizations mainly used file processing systems.
• Data was stored in separate files created for individual applications.
2] Hierarchical and Network Databases - 1960s → 1970s
• To overcome some limitations of file systems, early database models were developed.
• Hierarchical Database :- Data was organized in a tree-like structure using parent-child relationships.
• Network Database :- The network model allowed records to have more complex relationships, including many-to-many relationships.
These models were useful but could be complex to design and maintain.

3] Relational Databases – 1970s
↳ The relational database model was introduced by E.F. Codd in 1970.
↳ It represented data using tables consisting of rows & cols.
↳ The model became extremely popular because it provided a simple and flexible way of organizing data.

4] Object-Oriented Databases – 1980s → 1990s
↳ With the growth of OOP, object-oriented database systems were developed.
↳ They could store data in the form of objects, similar to programming languages such as Java and C++.

5] Web and client-Server Database Applications – 1990s
↳ The growth of Internet and World wide Web greatly increased the use of databases.
↳ Web applications began using databases to store and retrieve information.

6] Distributed and Cloud Databases – 2000s
↳ Distributed databases store data across multiple locations or systems.
↳ Later, Cloud Computing made it possible to use databases through cloud infrastructure.

7] Big Data and NoSQL Databases – 2000s → 2010s
↳ Traditional relational databases were not always ideal for every large-scale or rapidly changing workload.
↳ This led to the growth of NoSQL databases, such as:
• MongoDB • Cassandra • Redis

8] Modern Database Applications – Present
↳ Today, databases are used with technologies such as:
• cloud computing • Big data • AI • ML • IoT
↳ Modern systems may use both SQL and NoSQL databases, depending on the applications requirements.

5] When Not to Use a DBMS.

=> A DBMS is a useful for storing, managing and retrieving large amounts of data. However, using a DBMS is not always necessary. In some situations, a simple file system or other storage method is more suitable.

1] Small and Simple Applications.
- If an application handles only a small amount of data and has simple requirements, using a DBMS may be unnecessary.

2] High Initial Cost
- Setting up a DBMS may require costs for software, hardware, installation, configuration and maintenance. For a small organization, these costs may not be justified.

3] Need for Specialized Hardware.
- Some, DBMS applications require powerful servers, sufficient storage and backup systems. If such resources are unavailable, a DBMS may not be appropriate

4] Simple Data Storage Requirements.
- If data only needs to be stored and read without complex searching, updating or relationships, ordinary files may be sufficient.

5] Real-Time Applications with very Fast Response.
- Some applications require extremely fast and predictable responses. The overhead of a DBMS may sometimes

affect performance. In such cases, specialized storage systems may be preferred.

6] Embedded or Small Devices.
- Devices with limited memory, processing power, and storage may not be suitable for a full DBMS.

7] No Requirement for Multiple Users.
- DBMS provide features such as concurrency control and transaction management for multiple users. If only one person uses the data, these features may not be necessary.

8] Temporary Data.
- For data that is required only for a short period and does not need permanent storage, using a DBMS can be unnecessary.

9] Simple Applications with Limited Data Sharing
- If data does not need to be shared between different applications or users, a local file-based system may be easier and cheaper to maintain.

10] When DBMS Features Are Not Required
- A DBMS provides security, backup, recovery, transaction, data sharing and integrity management. If an application does not require these features, using a DBMS can add unnecessary complexity.

6] Relational Model Concepts.

⇒ The Relational Model is one of the most widely used database models. It represents data in the form of tables (relations) consisting of rows and columns. It was proposed by E.F. Codd.

1] Relation (Table)
↳ A relation is a table used to store data about a particular entity or concept.
↳ Eg :- Student table.

Sid	Name	City	Age
101	Rahul	Surat	22
102	Riya	Pune	24

Here, Student table is a relation.

2] Tuple (Row)
↳ A tuple represents a single record in a relation.
↳ For Eg :- (101, Rahul, Surat, 22) is one tuple of the Student relation.

3] Attribute (Column)
↳ An attribute represents a property or characteristic of an entity.
↳ In student table, Sid, Name, City and Age are attributes.

4] Domain
↳ A domain is the set of valid values that an attribute can contain
↳ For Eg :- • Age - integers from 1 to 100.
• Sid - Positive integers.

Thus, a domain defines what type of values are allowed.

5] Degree.
↳ The degree of a relation is the number of attributes (columns) in the table.
↳ For student table:- Sid, Name, City, Age.
There are 4 attributes
∴ Degree = 4.

6] Primary key.
↳ A primary key is an attribute or combination of attributes that uniquely identifies each tuple in a relation.
↳ Eg:- Sid can be the primary key because every student has a unique ID.

7] Foreign key
↳ A foreign key is an attribute in one table that refers to the primary key of another table.
↳ Eg:- Student Table

Sid	Name	Course_id
101	Rahul	C01
102	Riya	C02

Course Table

Cid	Course_name
C01	MSc IT
C02	BCA

Here, Cid in the student table is a foreign key referring to Cid in the Course table.

8] Integrity Constraints.
↳ Integrity constraints are rules that maintains the accuracy and consistency of data.
↳ Important types include:-
• Entity Integrity:- Primary key cannot be NULL
• Referential Integrity:- Foreign key must refer to a valid

record in the referenced table.
- Domain Integrity :- Values must belong to the defined domain.

9] Relationships Between Tables.
↳ Tables can be related to each other using keys.
↳ Common relationships are :-
- One-to-One (1:1)
- One-to-Many (1:N)
- Many-to-Many (M:N)

7] Relational Model Constraints and Schemas.

⇒ The relational model represents data in the form of tables called relations. To maintain the correctness, consistency and structure of data, it uses constraints and schemas.

(i) Relational Model Constraints.
↳ Constraints are rules that specify what values can be stored in a database. They help maintain the accuracy and integrity of data.
(A.) Domain Constraint.
- A domain constraint specifies the valid values and data type for an attribute.

(B) Key Constraint
- A key constraint ensures that certain attributes can uniquely identify a record.

(C) Entity Integrity Constraint.
- The entity integrity constraint states that a primary key cannot be NULL.

(D) Referential Integrity Constraint
- Referential integrity maintains Consistency between two related tables.

(E) General / Busines Constraints
- These are rules based on the requirements of an organization or applications.

(2) Relational Schema.
↳ A relational schema describes the structure of a relation/table. It specifies the table name, attributes and their domains.
↳ Eg :- Student (sid, Name, Age, City)
Here, Student - Relation name.
sid, Name, Age, City → Attributes.
A schema describes the design/structure of the table, not the actual records stored in it.
↳ Schema :- Student (sid, Name, Age, City)
↳ Instance :-

sid	Name	Age	City
101	Rahul	22	Surat
102	Riya	24	Pune

The schema remains generally fixed, while the instance changes when records are inserted, updated or deleted.

(3) Database Schema
↳ A database schema is the overall logical structure of the entire database. It contains multiple relational schemas and defines relationships between them.

↳ For Example :-
student (sid, Name, dept_id)

Department (Dept_id, d_name)
Course (C_id, C_name)
The database schema defines how these tables are organized and related.

8] Update Operations and Transactions.

⇒ In a relational database, update operations are used to add, modify or remove data, while transactions are used to perform a group of database operations as a single logical unit.

1] Update Operations.

↳ Update operations are actions that change the data stored in a database. The main update operations are

(A.) Insert Operation

- The insert operation is used to add a new record to a table.
- Eg:- Insert into student values(101, 'Rahul', 21, 'Surat');
- This adds a new student record to the student table

(B) Delete Operation

- The delete operations is used to remove existing recording from a table.
- Eg:- Delete from student where sid = 101;
- This removes the student whose ID is 101.

(C) Update / Modify Operation

- The update operation is used to modify existing data
- Eg:- Update student set City = 'Pune' where sid = 101;

- This changes the city of student id1 from Mumbai to Thane.

↳ Problems Arising Update Operations.
- It must follow database constraints. Otherwise, they can cause inconsistencies.
- For Eg :- if a foreign key refers to a department that does not exist, referential integrity will be violated. Similarly, inserting a duplicate primary key can violate a key constraint.
- ∴ The DBMS checks constraints whenever data is inserted, deleted or updated.

2] Transactions.
↳ A transactions is a sequence of one or more database operations that are treated as one logical unit of work.

↳ For Eg:- transferring £5,000 from Account A to Account B involves:-
(i) Deduct £5,000 from Account A.
(ii) Add £5,000 to Account B.
Both operations should be completed successfully. If one operation fails, the others should also be cancelled.

↳ A transaction follows four important properties called ACID
(i) Atomicity :- A transaction is treated as all or nothing. Either all operations are completed or none of them are applied.

(ii) Consistency :- A transaction must take the database from one valid state to another valid state while

maintaining all Constraints.

(iii) Isolation :- Multiple transactions running at the same time should not improperly interfere with each other.

(iv) Durability :- Once a transaction is committed, its changes are permanently stored, even if a system failure occurs afterward.

↳ Commit :- It permanently saves the changes made by a transaction.

↳ Rollback :- It cancels the changes made during the transaction and restores the previous state.

9] Dealing with Constraint Violations.

⇒ In a relational database, constraints are rules that maintain the correctness and consistency of data. Sometimes an INSERT, UPDATE or DELETE operation may violate these rules. Such a situation is called a constraint violation. A DBMS detects the violation and takes appropriate action to maintain database consistency.

↳ Types of Constraint Violations:-

(i) Domain Constraint Violation

- This occurs when a value does not belong to the valid domain of an attribute.

(ii) Key Constraint Violation

- A key constraint is violated when a duplicate value is entered for a primary key or unique key.

(iii) Entity Integrity Violation
- Entity Integrity requires that a primary key cannot be NULL.

(iv) Referential Integrity Violation.
- This occurs when a foreign key refers to a record that does not exist in the referenced table.

↳ How DBMS deals with Constraint Violations.

(i) Reject the Operation
- The simplest approach is to reject the INSERT, UPDATE or DELETE operation that violates a constraint.
- The DBMS displays an error message and does not apply the invalid operation.

(ii) Rollback the Transaction
- If a violation occurs during a transaction, the DBMS can Rollback the transactions.
- This cancels the changes made by the transaction and restores the previous consistent state.

(iii) Cascade the Operation.
- For some foreign-key relationships, the DBMS can automatically apply the operation to related records. (ON DELETE CASCADE)

(iv) Set NULL
- In some situations, when the referenced record

is deleted, the foreign key value can be set to NULL.
(ON DELETE SET NULL)

(v) set a Default Value
- A foreign key or other attribute can sometimes be assigned a predefined default value when the referenced record is removed.
(ON DELETE SET DEFAULT).

↳ Constraints are checked during database operations to ensure that the database remains consistent.

↳ Proper handling of constraint violations helps to:-
- Maintain data accuracy.
- Prevent invalid data.
- Maintain referential integrity.
- Avoid duplicate records.
- Maintain database consistency.
- Protect relationships between tables.
- Recover safely from transaction failures.

10] Practice Session with Example Queries and Exercises.

=) A practice session in a Relational Database Management System helps students understand how to create tables, insert data, retrieve records, update information, delete records and use constraints. SQL (Structured Query Language) is commonly used for these operations.

↳ Consider a student table,

sid	Name	Age	City	Marks
101	Rahul	20	Navsari	85
102	Riya	22	Mumbai	90
103	Amit	21	Surat	80

(i) Creating table:- The create table command is used to create a new table.

Syntax:- Create table student (
sid int primary key,
Name varchar(15),
Age int,
City varchar(15),
Marks int);

(ii) Inserting Records :- The insert Command adds records to the table.

Syntax:- insert into student values (101, "Rahul", 20, "Navsari", 85);

(iii) Retrieving Records :- The select command is used to retrieve data.

Syntax:- select * from student;

(iv) Using where clause :- Where is used to retrieve records that satisfy a Condition.

Syntax:- select * from student where Marks > 80;

(v) Updating Records :- The update Command modifies existing data.

Syntax :- Update student set City = "Ahmedabad" where sid = 102;

(vi) Deleting Records :- The delete Command removes records.

Syntax :- Delete from student where sid = 103;

(vii) Using Aggregate functions :- SQL provides functions such as COUNT(), SUM(), AVG(), MAX(), MIN().

Syntax:- Select AVG (Marks) from student;
Select SUM (Marks) from student;
Select MAX (Marks) from student;

(viii) Sorting Records :- The order by clause is used to sort results.

Syntax :- Select * from student order by Marks DESC;

Eg:- 1] Display all students from Mumbai.
=> select * from students where city = "Mumbai";

2] Display students whose marks are greater than 80.
=> Select * from students where Marks > 80;

3] Find the lowest marks.
=> select MIN (Marks) from student;

4] Find the average age of students.
=> Select AVG (Age) from students.

5] Display students in ascending order of Name.
=> select * from student order by Name asc;

6] Change Amit's marks to 82.
=> Update student set Marks = 82 where Sid=103;

7] Delete the student whose ID is 104.
=> Delete from student where Sid = 104;

8] Count the total number of students.
=> select Count (Sid) from students;